06-6-egress-limitations

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 6 supporting write-up: which layer the egress control sits at, the layers rejected and why, and seven named things it still fails to stop.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/05-egress.md`

Egress control — permit one KYC provider, deny everything else

Threat **T5** (bulk PII exfiltration) and the outbound half of **T2** (C2 channel from a compromised pod). Artefacts: `k8s/networkpolicy-egress.yaml`, `k8s/istio-egress.yaml`.

Which layer, and why not the others

I control egress at **two** layers, because neither alone is a control:

layer	what it can express	why it is not sufficient alone
NetworkPolicy (CNI, L3/L4)	"this pod may open TCP/443 to these pods and these CIDRs"	cannot express a hostname. The KYC provider is SaaS behind a CDN; its IPs change without notice. Allowing today's `/24` means either a broken integration next month or an allowlist that includes every other tenant of that CDN.
Istio egress gateway + `REGISTRY_ONLY` (L7/SNI)	"only `kyc.example-kyc.com`, only from pods labelled `app=accounts-api`, only against this CA"	the sidecar is inside the pod's trust boundary. An attacker with code execution in the pod can, in principle, talk to the sidecar's admin port or bypass the iptables redirect.

Together: the NetworkPolicy makes the egress gateway the **only** route off the pod, and the gateway makes `kyc.example-kyc.com` the only destination through it. Compromising one leaves the other standing.

Layers I considered and rejected:

- **A NAT-gateway allowlist / AWS Network Firewall at the VPC edge.** Correct place for a whole-VPC policy, wrong place for this: it cannot distinguish `accounts-api` from the twelve other services sharing the NAT gateway, so the rule would have to permit the union of everyone's destinations. It is a good *second* net and I would run it — as a coarse deny-list, not as this control.
- **Squid/Envoy forward proxy with explicit `HTTP_PROXY`.** Strictly better visibility (full URL, not just SNI). Rejected because it requires the application to honour the proxy variables, and an attacker's code simply does not — so it controls the honest traffic and not the dishonest kind, which is the wrong way round.

The `hostNetwork: false` Kyverno policy is a hard dependency of all of this. A pod on the host network is not on the pod network, so no NetworkPolicy applies to it, and this entire control silently evaporates. That is why `disallow-host-namespaces` ships in **Enforce** on day one while softer policies start in Audit.

What it still fails to stop

Honestly, and in descending order of how much it would bother me:

1. **Exfiltration through the permitted channel.** The single largest gap. An attacker who can call the KYC provider can send customer records *as* KYC requests — the destination is allowed, the port is allowed, the TLS is valid. Nothing in this design inspects the payload. Partial mitigations, none of them complete: per-pod egress byte-rate alerting (a bulk dump looks nothing like the normal request profile), and the RDS-side detection of an unbounded `SELECT` in the runbook. **The real fix is not a network control at all** — it is field-level access control in the application so that the KYC path can only read the four fields it needs. Named in `DECISIONS.md` §4 as the detection gap I would close first.
2. **DNS tunnelling.** DNS is restricted to CoreDNS pods rather than UDP/53 anywhere, which is better than most policies get. But CoreDNS forwards recursively, so an attacker who can resolve `<base64-chunk>.evil.example` still gets data out at a few hundred bytes per query. Closing it needs a CoreDNS policy plugin or a resolver allowlist; not built, and cheap to add.
3. **The sidecar bypass.** Code executing as the pod's UID can attempt to reach the Envoy admin interface or manipulate the iptables redirect. Istio's `ISTIO_META_DNS_CAPTURE` and dropping `NET_ADMIN` (already done — `drop: ALL`) raise the bar; they do not eliminate it.
4. **A compromise of the KYC provider itself.** The traffic is permitted and the certificate is valid because the destination genuinely is them. Explicitly accepted in the threat model, with field minimisation as the compensating control.
5. **Anything on the node.** `hostNetwork` is blocked for pods, but a co-tenant's container escape reaches the node's network namespace, where none of this applies. That is the consequence I accepted when I called container escape the overrated threat, and I stand by it — but it is the consequence.
6. **The ingress side of a data pull.** These policies stop the pod from *sending*. An attacker who can query the API through the ALB with valid credentials and page through every account is not touching egress at all. That is the WAF's and the application's rate limiting to catch, and neither is in this repo.

The compensating detection for (1) and (2) is `accounts_api_unexpected_egress` in `../runtime/falco-rules.yaml` — which, being port-based, catches (2) only if the tunnel leaves the permitted port set. It does not catch (1) at all. Saying so is more useful than a diagram with a green tick on it.